UNIVERSITY OF PENNSYLVANIA

ESE 650: LEARNING IN ROBOTICS

HOMEWORK 3

Changelog: This space will be used to note down updates/errata to the homework problems.

Read the following instructions carefully before beginning to work on the homework.

- You will submit neatly written solutions via Gradescope. You can use LaTeX (encouraged) but you can also write by hand. You can use hw_template.tex on Canvas in the "hw" folder to do so. If your handwriting is *unambiguously legible*, you can submit PDF scans/tablet-created PDFs.
- Please start a new problem on a fresh page and mark all the pages corresponding to each problem. Failure to do so may result in your work not graded completely.
- Clearly indicate the name and Penn email ID on your submitted solutions.
- For each problem in the homework, you should mention the total amount of time you spent on it.
- You can be informal while typesetting the solutions, e.g., if you want to draw a picture feel free to draw it on paper clearly, click a picture and include it in your solution. Do not spend undue time on typesetting solutions.
- You will see an entry of the form "HW 0 PDF" where you will upload the PDF of your solutions. You will also see entries like "HW 0 Problem 1 Code" where you will upload your solution for the respective problems. **For each programming problem, you should create a fresh Python file**. This file should contain all the code to reproduce the results of the problem and you will upload the .py file to Gradescope. If we have installed Autograder for a particular problem, you will use the Autograder. Name your file to be "pennkey_hw0_problem1.py", e.g., I will name my code for Problem 1 as "pratikac_hw0_problem1.py".
- **You should include all the relevant plots in the PDF, without doing so you will not get full credit.** You can, for instance, export your Jupyter

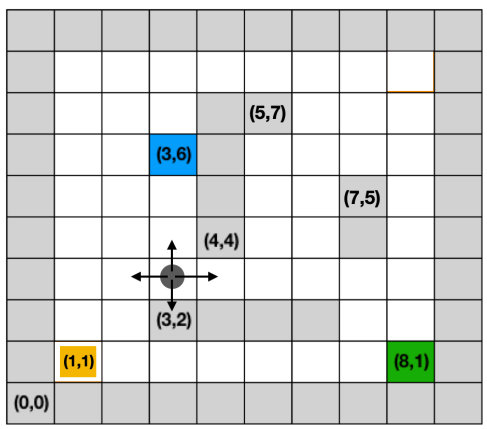
notebook as a PDF (you can also use text cells to write your solutions) and export the same notebook as a Python file to upload your code.

- **Your PDF solutions should be completely self-contained. If the question requires you to produce a plot, you must have it in the PDF to get credit. We will run the Python file to check if your solution reproduces the results in the PDF.**

Credit. The points for the problems add up to 120. You only need to solve for 100 points to get full credit, i.e., your final score will be $\min(\text{your total points}, 100)$.

Problem 1 (Policy Iteration, 20 points (No Autograder)). Consider the following Markov Decision Process. The state-space is a 10×10 grid, cells that are obstacles are marked in gray. The initial state of the robot is in blue and our desired terminal state is in green. The robot gets a *reward* of 10 if it reaches the desired terminal state with a discount factor of 0.9. At each non-obstacle cell, the robot can attempt to move to any of the immediate neighboring cells using one of the four controls (North, East, West and South). The robot cannot move diagonally. The move succeeds with probability 0.7 and with the remainder probability of 0.3, the robot can end up at some other cell. In short,

$$\begin{aligned}
\mathrm{P}(\text{moves north} \mid \text{control is north}) &= 0.7, \\
\mathrm{P}(\text{moves west} \mid \text{control is north}) &= 0.1, \\
\mathrm{P}(\text{moves east} \mid \text{control is north}) &= 0.1, \\
\mathrm{P}(\text{does not move} \mid \text{control is north}) &= 0.1.
\end{aligned}$$



Similarly, if the robot desired to go east, it may end up in the cells to its north, south, or stay put at the original cell with total probability 0.3 and actually move to the cell east with probability 0.7. The robot pays a cost of 1 (i.e., reward is -1) for each control input it takes, regardless of the outcome. If the robot ends up at a state marked as an obstacle (all grey cells are obstacles, i.e., cell marked (0,0), (0,1), (3,2) etc. are obstacles), it gets a reward of -10 for each time-step that it remains inside the obstacle cell. The robot is allowed to stay in the goal state indefinitely (i.e., take a special action to "not move") and this action gets no reward/cost.

We would like to implement policy iteration to find the best trajectory for the robot to go from the blue cell to the green cell.

(a) **(0 points)** Carefully code up the above environment to run policy iteration. You will need to think about how to code up the probability transition matrix

$\mathbb{R}^{100\times 100} \ni T_{x,x'}(u) = \mathrm{P}(x' \mid x, u)$, the run-time cost $q(x, u)$, and the terminal cost $q_f(x)$. Policy iteration is easy to implement if you represent all the above quantities as matrices and vectors. Plot the environment to check if it confirms to the above picture.

(b) **(10 points)** Initialize policy iteration with a feedback control $u^{(0)}(x)$ where the robot always goes east, this results in a policy $\pi^{(0)} = (u^{(0)}(\cdot), u^{(0)}(\cdot), \ldots)$. Write the code for policy evaluation to obtain the cost-to-go from every cell in the above picture for this initial policy. Plot the value function $J^{\pi^{(0)}}(x)$ as a heatmap in the above picture.

(c) **(10 points)** Execute the policy iteration algorithm, you will iteratively perform policy evaluation and policy improvement steps. For the first 4 iterations, plot the feedback control $u^{(k)}(x)$ (using arrows as shown in the lecture notes (https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.arrow.html, you can also write the control input in the cell). You should color the cell using the value function $J^{\pi^{(k)}}(x)$.

We have left the transition probabilities and the reward structure a bit vague to force you to think carefully of the nuances of this problem. But some clarification could be useful.

(1) You can code up what are called "sticky obstacles", i.e., if the robot enters an obstacle, then it stays there forever while incurring the obstacle cost at each time instant.

(2) It is easiest to think of the runtime cost in this problem as a function of three quantities $q(x, u, x')$ where x is the current state, u is the control and x' is the next state. The Bellman equation the becomes

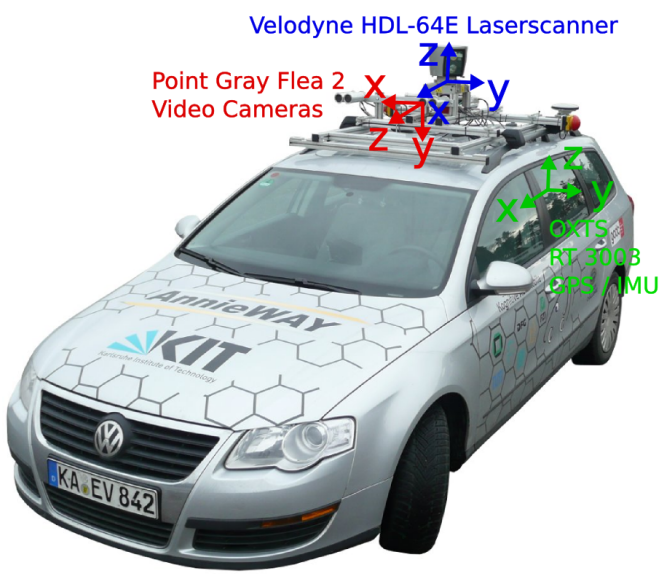
$$J^*(x) = \min_{u \in U} \mathop{\mathrm{E}}_{x'} \left[q(x, u, x') + \gamma J^*(x') \right].$$

You will submit your own code for this problem, there is no Autograder.

Problem 2 (Simultaneous Localization and Mapping (SLAM) with a particle filter, 60 points (No Autograder).).

Download the data from the following link
https://drive.google.com/file/d/1X92V8ISgV50KckK199v0BW_WGGRbQ1Ct/view?usp=sharing.

In this problem, we will implement mapping and localization in an outdoor environment using information from a GPS/IMU system and a LiDAR sensor. We have provided you with a subset of the KITTI visual odometry dataset collected from a laserscanner mounted on a Volkswagen Passat B6. You can read more about the hardware setup on the website (https://www.cvlibs.net/datasets/kitti/setup.php), original paper (https://www.cvlibs.net/publications/Geiger2012CVPR.pdf), and the video (https://www.youtube.com/watch?v=KXpZ6B1YB_k).

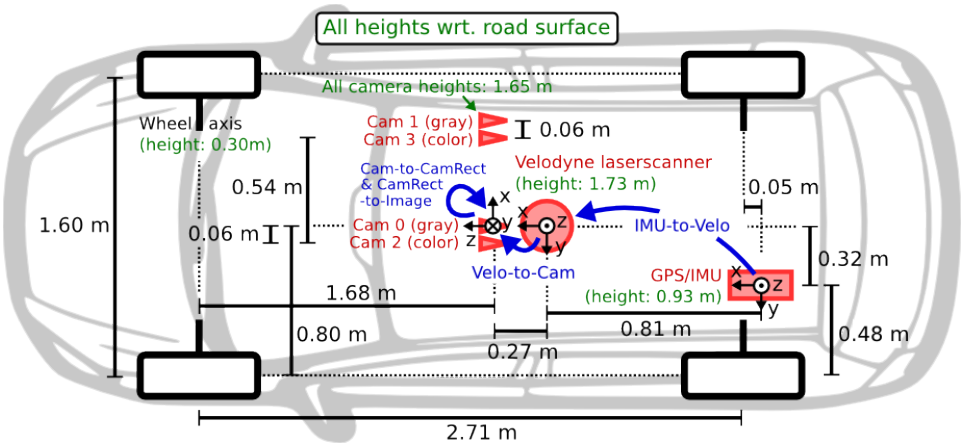


Hardware setup of 'Annieway'. The autonomous driving platform (Annieway) has a Velodyne HDL-64E laser scanner mounted on the roof, which is 1.73m above the ground. The laser scanner has a vertical resolution of 64 and rotates at 10 frames per second, capturing approximately 100,000 points per rotation. Additionally, the autonomous driving platform is equipped with 2 color and grayscale cameras with each color and grayscale camera mounted on the center and right side of the car.

The second type of observations we will utilize corresponds to the car's location. However, in contrast to the previous homework where we used the raw accelerometer and gyroscope readings to get the orientation, we will use the (x, y, z) pose of

the center camera. These poses were created by the GPS/IMU system with RTK float/integer corrections (read up on wikipedia on what RTK is) enabled. We will transform the poses to coordinate system (x, z, θ). Conceptually, these RTK poses are the same as Vicon in the previous homework, these are a precise estimate of the car's pose that we will use to check how well our SLAM system is working.

Coordinate frames. Poses are in coordinate system of the the left camera which has the Z axis pointing forwards, the X axis pointing right and the Y axis pointing downwards (see the picture below). However, the LiDAR has a coordinate frame with the X axis pointing forwards, Y axis pointing left and the Z axis pointing upwards. The camera is at a height of 1.65m and the height of the LiDAR is 1.73m above the ground. The world coordinate frame where we want to build the map has its origin on the X-Z ground plane.



Data and code.

(a) **(0 points)** We have provided you 4 datasets corresponding to 4 different trajectories of the car driving around the mid-size city of Karlsruhe, in rural areas and on highways. We have modified the original KITTI dataset by adjusting the LiDAR frequency from 10Hz to 2Hz (so that you do not have to process very large point clouds). The datasets are labeled as 00, 01, 02 and 03 in our modified version, they correspond to trajectories 02, 06, 07 and 08 from the original KITTI dataset, respectively. The provided dataset consists of three main directories: **calib**, **odometry** and **poses**.

The **calib** folder contains a .txt file that provides calibration data for the cameras. The P0/P1/P2/P3 are the 3×4 projection matrices after rectification, where P0/P2 corresponds to the left cameras, and P1/P3 denotes the right cameras (refer to

figure above for details of the camera). The Tr transforms a point from Velodyne coordinates into the left rectified camera coordinate system. Additionally, the times.txt file stores timestamps for each image pair in seconds; however, this file will not be used for this homework. We will not use any observations from the cameras in this homework.

The **odometry** folder contains Velodyne point cloud data stored in a binary format to save space. Each scan is represented as an N$\times$4 floating-point matrix, where the first three values correspond to the x, y, and z coordinates, and the fourth value represents the reflectance. This data is organized in a row-aligned format, meaning that the first four values in the file correspond to the first recorded measurement. The function `load_kitti_lidar_data` in `load_data.py` is used to read the odometry data, while the functions `show_kitti_lidar` and `show_kitti_lidar_sequence` provide visualization of the KITTI dataset.

The **poses** folder contains trajectory poses for the entire sequence. Each .txt file stores an N$\times$12 table, where N represents the number of frames in the sequence. Each i row corresponds to the i-th pose of the left camera coordinate system (with the Z-axis pointing forward) represented by a 3$\times$4 transformation matrix. The transformation matrices are stored in a row-aligned format, meaning that the first entries correspond to the first row of the matrix. These matrices transform a point from the i-th coordinate system to the first (0th) coordinate system. Consequently, the translational component (the 3$\times$1 vector in the fourth column) represents the pose of the left camera coordinate system in the i-th frame relative to the first (0th) frame. The function `load_kitti_poses` in `load_data.py` is used to read the pose data, while the function `trajectory2d` in `load_data.py` provides a visualization of the trajectory.

You should read these functions carefully and check the values returned by them.

(b) **(0 points)** Next look at the `slam.py` file provided to you. Read the code for the class `map_t` and `slam_t` and the comments provided in the code very carefully. You are in charge of filling in the missing pieces marked as TODO: XXXXXX. A suggested order for studying this code is as follows: `slam_t.read_data`, `slam_t.init_particles`, `slam_t.lidar2world`, `map_t.__init__`, `map_t.grid_cell_from_xz`. Next, the file `utils.py` contains a few standard rigid-body transformations that you will need. You should pay attention to the functions `smart_plus_2d` and `smart_minus_2d` that will be used to code up the dynamics propagation step of the particle filter.

(c) **(10 points, dynamics step)** Next look at `main.py` which has two functions `run_dynamics_step` and `run_observation_step` which act as test functions to

check if the particle filter and occupancy grid update has been updated correctly. The `run_dynamics` function plots the trajectory of the car (as given by its pose data). It also initializes 3 particles and plots all particles at different time-steps while performing the dynamics step with a very small dynamics noise; this is a very neat way of checking if dynamics propagation in the particle filter is working correctly. This function will create two plots, one for the odometry trajectory and one more for the particle trajectories, both these trajectories should match after you code up the dynamics function `slam_t.dynamics_step` correctly.

(d) **(20 points, observation step)** The function `run_observation_step` is used to perform the observation step of the particle filter to get an estimate of the location of the robot and updates to the occupancy grid using observations from the LiDAR. First read the comments for the function `slam_t.observation_step` carefully. We first discuss the particle filtering part.

We first discuss the particle filtering part.

(i) For each particle, assuming that the particle is indeed the true position of the car, transform the LiDAR scanned point clouds into the world coordinates using the `slam_t.lidar2world` function.

(ii) In order to compute the updated weights of the particle, we need to know the likelihood of LiDAR scans given the state (our current occupancy grid in the case of SLAM). We are going to use a simple model to do so

$$\log \mathrm{P}(\text{LiDAR scan as if the car is at particle } p \mid m) = \sum_{ij \in O} m_{ij} \tag{1}$$

where O is the set of occupied cells as detected by the LiDAR scan assuming the robot is at particle p and m_{ij} is our current estimate of the binarized map (more on this below). In simple words, if the occupied cells as given by our LiDAR match the occupied cells in the binarized map created from the past observations, then we say the log-probability of particle p is large.

(iii) You will next implement the function `slam_t.update_weights` that takes the log-probability of each particle p, its previous weights, calculates the updated weights of the particles.

(iv) Typically, resampling step (`slam_t.stratified_resampling`) is performed only if the effective number of particles (as computed in `slam_t.resample_particles`) falls below a certain threshold (30% in the code). Implement resampling as we discussed in the lecture notes.

Mapping. We have a number of particles $p^i = (x^i, z^i, \theta^i)$ that together give an estimate of the distribution of the location of the car. For this homework, you will only use the particle with the largest weight to update the map although typically

we update the map using all particles. Our goal is simple: we want to increase the `map_t.log_odds` array at cells that are recorded as obstacles by the LiDAR and decrease the values in all other cells. You should add `slam_t.log_odds_occ` to all occupied cells and add `slam_t.log_odds_free` from all cells in the map. It is also a good idea to clip the `log_odds` to like between [-`slam_t.map.log_odds_max`, `slam_t.map.log_odds_max`] to prevent increasingly large values in the `log_odds` array. The array `slam_t.map.cells` is a binarized version of the map (which is used above to calculate the observation likelihood).

Check the `run_observation_step` function after you have implemented the observation step.

(e) Since the map is initialized to zero at the beginning of SLAM which results in all observation log-likelihoods to be zero in [1], we need to do something special for the first step. We will use the first entry in `slam_t.poses` to get an accurate pose for the robot and use its corresponding LiDAR readings to initialize the occupancy grid. You can do this easily by initializing the particle filter to have just one particle and simply calling the `slam_t.observation_step` as shown in `main.py`.

(f) **(30 points)** You will now run the full SLAM algorithm that performs one dynamics step and observation step at each iteration in the function `run_slam` in `main.py`. Make sure to start SLAM only after the time when you have both LiDAR scans and joint readings (the two arrays start at different times). For all 4 datasets, you will plot the final binarized version of the map, (x, z) location of the particle in the particle filter with the largest weight at each time-step and the odometry trajectory (x, z) (in a different color); this counts for 10 points each.

Some Notes. This problem is much easier and shorter than it may seem. You should go through these steps carefully and in the suggested order. You should make sure that the results of the previous step are correct before proceeding. The two functions in main.py to check the dynamics and observation step are very important to find bugs. You do not need to implement more than 100 lines of code.

Problem 3 (Building a NeRF, 40 points (No Autograder; You can implement this on Google Colab if necessary)). NeRF is a technique for mapping complex scenes by optimizing an underlying continuous volumetric representation using a sparse set of input views. NeRFs represent the scene using a fully connected (non-convolutional) deep network. The input to the network is 5-dimensional $x \in \mathrm{SE}(3)$ (without the roll). This consists of the 3-dimensional location in Euclidean space, and two viewing directions. Using this input, the neural network inside the NeRF outputs volume density $\sigma(x)$ and a view-dependent color $c(x)$ at that spatial location. In this problem, we will implement a simplified NeRF, which only takes 3D Euclidean coordinates (as you can imagine, the pictures from such a NeRF do not change depending upon the viewpoint and therefore they will not look as natural). We will implement the simplest possible version of a NeRF without a lot of bells and whistles that are used in actual implementations, on downsized training images. This way, the model will be small enough to train locally on your laptop, or on Google Colab.

(a) Data Loading and COLMAP (5 points). On Canvas, we provide a dataset consisting of 100 LEGO images captured from various angles (you are also encouraged to capture your own dataset and show results on it). You will use COLMAP, a Structure-from-Motion (SfM), and Multi-View Stereo (MVS) pipeline to obtain camera extrinsic estimation. COLMAP is an open-source library that is compatible with Mac (install using Homebrew), Linux, and Windows. Install COLMAP first following instructions provided in the COLMAP documentation or the one that NeRF Studio provides.

Assuming the images are taken by the same camera (images have the same intrinsic parameters, i.e., the same camera calibration), you should use COLMAP to reconstruct a sparse model. The package also comes with a GUI (you can call it using "colmap gui") that provides a great interface and visualization. After getting the sparse model, you will have to understand the provided colmap2nerf script and use it to transform the sparse model file into a JSON file which contains information such as camera intrinsic and extrinsic corresponding to each image. We will need this information to begin training the NeRF.

To simplify this task, we have also provided the correct JSON file on Canvas. You are welcome to use it if you do not want to run COLAMP to get the camera intrinsics and extrinsics. You will not get these 5 points, but this way you will be able to proceed with the rest of the problem.

You will then implement the **load_colmap_data** function, which reads in the generated JSON file as well as the raw images. We recommend you resize the raw images to a lower resolution, for example, from 800×800 to 200×200, so that it

is feasible to train everything on your laptop. After resizing the images, remember to change the camera parameters (height, width, and focal length) accordingly. You should report these parameters in the PDF and how you calculated them.

(b) Implementation of the NeRF (20 points). You will now implement four key functions.

The get_rays function. Assuming a pinhole camera model, complete the **get_rays** function, which takes camera intrinsic parameters (camera calibration matrix) and extrinsic parameters (locations from where the images where collected) as input and returns a set of rays in the world frame. Each ray starts from the camera origin and passes through one of the pixels (see the figure in Section 4.5.1 in the lecture notes). We will use the homogeneous coordinates. Given a point $x_c = (i, j, k, 1)$ in the camera frame, the point can be transformed from the camera frame to the world frame with $x_w = T_w^c \, x_c$, where T_w^c denotes the 4×4 transformation matrix obtained from the previous question.

It is useful to emphasize the coordinate convention. We will adhere to the standard NeRF coordinate convention for camera coordinates: +X is right, +Y is up, and +Z points back and away from the camera, i.e., the -Z direction corresponds to the direction at which the looking at. It is important to note that other code-bases on the Internet may adopt the COLMAP/OpenCV convention, where the Y and Z axes are flipped compared to ours, but the +X axis remains the same. The world coordinate system is oriented such that the up vector is +Z. The XY plane is parallel to the ground plane.

The sample_points_from_rays function. Given a set of rays emanating from the camera center, we will discretize each ray into segments to approximate the integrals during volume rendering. Implement the sample_points_from_rays function, which returns an array of N_{sample} points along each ray in world coordinates.

(a) With a rough estimate of the distance from the object to the camera, we can determine the clipping thresholds s_{near} (the distance of the nearest point of interest) and s_{far} (the distance of the farthest point of interest). Each ray will only be evaluated within the range of s_{near} and s_{far}, which defines the volume of interest. Given a fixed number of points N_{sample}, a small $s_{\text{far}} - s_{\text{near}}$ means that sampled points along the ray are closer to each other; this leads to a better estimation for the integral.

(b) You can sample uniformly along the ray. For enhanced performance, consider incorporating some randomness into the sampling process while ensuring that there is at least one point every $(s_{\text{far}} - s_{\text{near}})/N_{\text{sample}}$.

The position_encoding function. Like we discussed in the lecture, an MLP with a finite width and a certain number of layers may not be able to represent functions of arbitrarily high bandwidths which are necessary to get high-frequency textures. This leads to blurry images from the NeRF. A neat solution to this issue is to use a different representation for the inputs $x \in \mathbb{R}^3$. Instead of using x we use

$$\varphi(x) = (\sin\Big(2^k x_1\Big), \sin\Big(2^k x_2\Big), \sin\Big(2^k x_3\Big), \dots)_{k=0,\cdots,10}$$

where k is the frequency and we choose, say 10 different frequencies. The input layer of the MLP would therefore be 30- instead of 3-dimensional.

The volume_rendering function. Here you will implement the volume rendering function in Equation 4.31 in the lecture notes. In summary, given a ray with points at distances

$$s_i = s_{\text{near}} + \frac{i}{N_{\text{sample}}}(s_{\text{far}} - s_{\text{near}})$$

we will calculate

$$\text{opacity: } \alpha_j = 1 - e^{-\sigma(s_j)(s_{j+1} - s_j)}$$

$$\text{transmittance: } p(s_i) = \prod_{j=1}^{i-1}(1 - \alpha_j)$$

$$\text{color: } c = \sum_{i=1}^{N_{\text{sample}}} c(s_i)\ \alpha_i \prod_{j=1}^{i-1}(1 - \alpha_j).$$

for each ray corresponding to each pixel. Implement the volume_rendering function, which renders an RGB image using the predicted radiance field.

(d) Network Training (10 points). We provide the neural network architecture and a simple training loop for you to start. Fill in the **nerf_step_forward** function with your implementations of the functions above. Your report should mention the parameters for the **train** function, including s_{near}, s_{far}, and N_{sample}. Start with N_{sample} of 32 and the hidden dimension of the MLP h_{dim} of 32. With this setting, you should be able to train the network on a laptop CPU in about 20 minutes.

(i) We highly recommend rendering and visualizing the network prediction every few iterations (doing so is similar to calculating the validation loss after few epochs while training a standard neural network-based classifier). This is an easy way to assess the network's performance. You can select one of the poses from the training set or randomly select your own pose as the test pose. Then, render an RGB image at the test pose using **nerf_step_forward** and check if the rendered image makes sense.

(ii) If you have access to a GPU, or decide to use Colab, consider increasing N_{sample} and h_{dim}. Doing so should lead to improved results.

You should report a plot of the training loss as a function of the number of weight updates. You should report the final training loss, and for about 5-6 randomly sampled images from the training dataset, you should show the original image and the one rendered from the NeRF from the same viewpoint (this is reporting predictions of the network on the training samples).

(e) Inference (5 points). Take the trained network, randomly pick 5 viewpoints from as different poses as you can and report the rendered RGB images from these viewpoints. Try to find viewpoints where the NeRF is working well as well as ones where it is not.